FMoE x SASRec 연구 메모

Codex 공유용 요약 문서

주제: stage-wise behavior feature 를 활용한 feature-aware MoE routing 의 진단, 실험 설계, 개선 방향

한 줄 요약

- 지금 문제의 핵심은 feature 자체의 무용성이 아니라, 유효한 behavior feature 를 어떤 시간 스케일에서 어느 정도 자유도로 router 에 맡길지의 문제다.
- 현재 관찰은 macro > mid >> micro 의 비대칭성을 시사하며, fully learnable routing 보다 constrained / hybrid control 이 더 적합할 가능성이 높다.
- 따라서 expressiveness 확대보다 routing instability 진단, feature usefulness 검증, stage-specific control 설계가 먼저다.

1. 현재 문제 설정 요약

현재 모델은 SASRec 기반 SeqRec 구조 위에 FFN-MoE 를 삽입하고, user/session behavior feature 를 router 입력으로 사용한다. 입력 feature 는 macro / mid / micro 3 개 stage 로 나뉘며, 각 stage 는 의미적으로 4 개 group 을 가진다. 현재 주요 형태는 **SASRec layer -> macro FFN MoE -> SASRec layer -> mid FFN MoE -> SASRec layer -> micro FFN MoE** 이다.

- Router 입력: hidden state + stage feature embedding (scalar feature 들을 projection 하여 embedding 화 후 concat, MLP router 적용)
- Routing: 주로 dense routing 이 더 나았으며, top-k 는 아직 성능상 우위가 없었음
- Expert: 과거엔 stage 당 12 개(4 groups x 3 experts), 최근엔 더 단순하게 stage 당 4 개 정도 사용
- 데이터: cleaned KuaiRec(short-form)와 LastFM-1K, session-aware split(70/15/15), metric 은 MRR@20 / HR@10

2. 현재까지의 핵심 관찰

관찰 포인트	해석
Feature-aware vs hidden-only	feature-aware routing 이 hidden-only routing 보다는 대체로 나음
SASRec 대비	하지만 feature-aware MoE 도 pure SASRec 을 일관되게

	넘지는 못함
Stage importance	macro 는 유효, mid 는 약한 보조, micro 는 자주 중복적·불안정
Best clue	일부 세팅에서 macro learnable + mid/micro rule-based 가 가장 좋았고, SOTA 근처까지 간 적도 있음
Complex router	hierarchical / more expressive router / extra feature bias 등은 대체로 개선이 미미하거나 오히려 악화
Working hypothesis	현재 병목은 expressiveness 부족보다 instability / underconstrained routing / inductive bias 붕괴일 가능성이 큼

중간 결론

- macro feature 는 user/session regime 를 반영하는 비교적 안정적인 control signal 일 가능성이 높다.
- micro feature 는 SASRec 의 causal self-attention 이 이미 잘 처리하는 short-term context 와 중복될 수 있다.
- 따라서 모든 stage 에 동일한 수준의 learnable routing freedom 을 주는 설계는 오히려 비효율적일 수 있다.

3. 가장 가능성 높은 failure mode

3-1. feature 는 유효하지만, router 가 feature 를 너무 강한 제어 신호로 쓰고 있을 수 있음

현재 router 는 대략 **$p^{(s)} = \text{Router}_s([h_t ; \phi(f_s)])$** 형태다. 여기서 h_t 는 token-level hidden, f_s 는 stage-level summary feature 이다. 문제는 coarse 한 summary 를 fine-grained expert routing 에 바로 사용하면서, 유효한 feature 를 과도한 자유도로 학습시키고 있을 가능성이다.

3-2. macro 는 regime signal, micro 는 backbone 과 중복될 수 있음

- macro: 활동량, session gap, mainstreamness, diversity 처럼 느리게 변하는 user/session mode 를 반영
- micro: 최근 interval, switch rate, short window statistics 처럼 매우 local 하며 변동성이 큼
- 따라서 micro 는 $I(f_{micro} ; y \mid h_t)$가 이미 작아졌을 가능성이 있고, router 에 별도 자유도를 주면 noise 가 늘 수 있음

3-3. granularity mismatch: macro signal 인데 token 마다 route 가 흔들릴 수 있음

macro feature 는 user/session-level 요약인데, routing 이 token 마다 크게 흔들리면 stage prior 가 실제로는 hidden noise 에 끌려가는 셈이다. 이를 보려면 macro route jitter 를 다음처럼 정의해 볼 수 있다.

$$J_s = (1 / |S|) \Sigma_{seq} (1 / (T-1)) \Sigma_t \| p_t^{(s)} - p_{(t-1)}^{(s)} \|_1$$

macro 는 같은 session 내에서 J_macro 가 낮아야 자연스럽고, mid 는 같은 rolling window 내에서 비교적 안정적이어야 한다. micro 만 상대적으로 높은 jitter 를 허용하는 편이 자연스럽다.

3-4. group semantics 가 설명으로만 있고 계산 그래프상 제약이 약함

- 현재는 stage feature 전체를 embedding 하여 router 에 넣기 때문에, 실질적으로는 group-aware expert 보다 flat expert pool 에 가까움
- hierarchical 구조를 도입해도 group 선택과 within-group routing 모두 learnable 이면, 사람이 준 semantic grouping 이 다시 학습 속에서 무너질 수 있음

4. 반드시 남겨야 할 diagnostics

항목	예시 식 / 통계	해석 포인트
Expert usage	$U_e = (1/N) \Sigma_n p_{n,e}$	전체 routing mass 분포. 어떤 expert 가 실질적으로 얼마나 쓰이는지 확인
Top-1 usage	$T_e = (1/N) \Sigma 1[\mathrm{argmax}\ p_n = e]$	dense routing 에서도 dominant expert 빈도를 확인
Router entropy	$H_n = -(1/\log E) \Sigma_e p_{n,e} \log p_{n,e}$	샘플별 routing 의 sharpness. 낮으면 과도한 몰빵, 높으면 거의 평균화
Load imbalance	CV(U), max/min, Gini	global usage 치우침 정도. imbalance 는 collapse 보다 약한 상태
Collapse	$N_{eff} = \exp(-\Sigma_e U_e \log U_e)$	effective expert 수. expert 가 4 개인데 N_eff 가 1~2 면 collapse 의심
Route jitter	$J_s = \mathrm{avg}\ \lVert p_t^{(s)} - p_{(t-1)}^{(s)} \rVert_1$	특히 macro/mid 에서 granularity mismatch 여부 판단
Feature sensitivity	$S_f = E[\ JS(p(h,f), p(h, f_{shuffled}))\ ]$	feature 를 실제로 routing 이 얼마나 쓰는지 확인

실무적으로 꼭 추가할 로그

- stage 별 U_e / T_e / N_eff / CV(U) / entropy histogram
- feature quantile bucket 별 routing heatmap (feature-bucket x expert)
- expert output similarity: expert 들이 사실상 같은 함수를 배우는지 점검
- validation permutation importance: feature 를 섞었을 때 metric 과 route 가 어떻게 달라지는지 확인
- slice evaluation: high activity, long session gap, high diversity, mainstreamness, repeat ratio 등 regime 별로 SASRec 대비 gain 확인

5. 최소 ablation plan (핵심 8 개)

Run	설정	목적
Run 1	Pure SASRec	anchor baseline
Run 2	Parameter-matched dense SASRec	dense MoE 이득이 capacity 때문인지 분리
Run 3	Hidden-only MoE	feature 가 router 에 실제로 유효한지 확인
Run 4	Macro-only learnable	가장 유망한 stage 단독 가치 확인
Run 5	Macro + Mid learnable	mid 의 incremental value 확인
Run 6	Macro + Mid + Micro learnable	full learnable 구조의 실제 효과 점검
Run 7	Hybrid: macro learnable + mid heuristic(or modulation) + micro off/weak gate	현재 관찰과 가장 잘 맞는 constrained design 검증
Run 8	Non-MoE feature conditioning baseline (FiLM / AdaLN / gated bias)	mid/micro 에 MoE 가 과한지, modulation 이면 충분한지 확인

비교 해석의 핵심

- Run1 vs Run2: dense MoE 성능이 단순 파라미터 증가 효과인지 분리
- Run3 vs Run4: feature-aware routing 의 실질 가치 확인
- Run4 vs Run5 vs Run6: macro / mid / micro stage value 분해
- Run6 vs Run7: fully learnable 보다 constrained hybrid 가 나은지 판단
- Run7 vs Run8: short-horizon feature 는 MoE 보다 modulation 이 더 적절한지 확인

6. 우선순위 높은 구조 수정안

우선순위	수정안	핵심 형태	왜 필요한가
1 순위	Base SASRec FFN + routed residual delta	$y = FFN_base(h) + \alpha_s(f_s) * \Delta_MoE^{(s)}(h)$	MoE 가 backbone 을 대체하지 않고, 필요할 때만 correction 하게 하여 안정

			성 확보
2 순위	Stage-specific freedom	macro: learnable / mid: shared+small specialist 또는 modulation / micro: off-by-default	현재 관찰인 macro > mid >> micro 를 architecture 에 반영
3 순위	Heuristic-anchored routing	p_s = softmax(z_s + β_s log(q_s + ε)) 또는 KL(p_s || q_s)	rule-based prior 를 출발점으로 두고 학습이 얼마나 벗어날지만 조절
4 순위	Granularity-matched routing	macro route 는 session-level 또는 slow update 로 broadcast	stage feature 의 time scale 과 routing granularity 를 맞춤
5 순위	Micro 를 confidence-gated modulation 으로 축소	y = y + g_micro(f,v) * Δ_micro(h)	micro 는 독립 expert selector 보다 작은 local corrector 로 쓰는 편이 자연스러울 수 있음
6 순위	Shared expert 추가	y = E_shared(h) + Σ_e p_e E_e(h)	모든 token 에 필요한 공통 변환과 특수 correction 을 분리

7. feature group inventory (요약)

Macro stage

장기 user/session regime 를 요약하는 비교적 안정적인 feature. router 의 주된 control signal 후보.

Group	대표 feature
Retention	user activity frequency session gap new user 여부 average session length
Rhythm	session start time / weekday sinusoidal weekend 여부 average interaction interval historical session gap
Spectrum	category entropy unique categories top-1 category share

	switch ratio
Mainstream	popularity average popularity std popularity entropy top-1 popularity-bin share

Mid stage

최근 중간 길이 window 의 몰입/변화/탐색/반복 경향을 담는 보조 신호.

Group	대표 feature
Deep diver	window entropy top-1 category share interval std session elapsed time
Mood surfer	popularity drift entropy drift popularity average
Pacer	interval average acceleration session progress
Repeater	novelty ratio repeat ratio unique item count

Micro stage

아주 최근 local pattern 을 담는 짧은 window feature. SASRec attention 과 중복될 수 있어 보수적 사용이 적합할 수 있음.

Group	대표 feature
Impulse	last interval time interval average interval std window validity
Hopper	short-term switch rate unique category count

Binger	re-consumption 여부 unique item count
Trend	popularity average popularity drift

8. Codex 에게 넘길 핵심 작업 지시

추천 작업 순서

- 1) 기존 코드베이스에 stage 별 routing diagnostics 로깅 추가: usage, top-1, entropy, N_eff, CV(U), jitter, feature-bucket heatmap
- 2) 핵심 8-run ablation 을 재현 가능한 config 형태로 정리
- 3) Base FFN + routed residual delta 버전 구현
- 4) macro learnable / mid constrained / micro weak or off-by-default 버전 구현
- 5) heuristic-anchored routing 과 shared expert 버전을 가볍게 추가
- 6) 최종적으로 stage 별 slice evaluation 리포트까지 자동 생성

가장 중요한 메시지: “더 복잡한 router 를 찾는 것”보다 먼저, **어떤 stage feature 가 실제로 유용하고, 어느 stage 에서 instability 가 발생하며, 어떤 granularity 가 맞는지**를 관측 가능한 상태로 만드는 것이 우선이다.